PoSIRd usage

Daniela

2026-07-20

Contents

1 Introduction **1**

2 One simulation **1**
2.1 Set hyperparameters 1
2.2 Get changepoints 2
2.3 Generate data 3
2.4 PoSIRd confidence regions 4
2.5 Confidence regions based on usual chi2 quantile 5
2.6 Estimated changepoints 7

3 Monte Carlo evaluation of coverage **13**

1 Introduction

Aim of this vignette is to illustrate the usage and benefit of PoSIRd for simultaneously valid confidence region for the mean of a 2-dimensional signal. To this aim, the workflow is

- Simulate two dimensional gaussian data with known covariance and piecewise constant mean
- Estimate the changepoints locations
- Get posir confidence region
- Get naive confidence region comparing with the $(1-\alpha)$ quantile of a χ^2_2 distribution
- Get naive confidence region comparing with the $(1-\alpha)$ quantile of a χ^2_2 distribution adjusted for the number of tests
- Compare the results in terms of coverage and volume of the ellypses

```
library(mvtnorm)
library(jointseg)
```

2 One simulation

2.1 Set hyperparameters

Get quantile table. *To be discussed:* in posir there are the functions posir_quantiles and load_quantiles that get the quantile table from the folder inst/extdata. Where should I put the new table, how should I call it, how should I modify the functions?

```
Qtable_path <- here::here(
  "Simulations",
  "1D_Discretization_20000_Gaussian_grid_1to.005by.005_signal_dim_2",
  "Table_quantiles.txt"
)
```

```r
message("Quantile path: ", Qtable_path)
```

```
## Quantile path: /home/dcorbett/github/posir/Simulations/1D_Discretization_20000_Gaussian_grid_1to.005
```

```r
if (!file.exists(Qtable_path)) {
  stop("Quantile table not found at: ", Qtable_path)
}

Qtable <- read_quantiles(Qtable_path)
```

For this simulations, the means will have possible states $(-2, 0, 2)$ in both dimensions. The covariance matrix is

$$\Sigma = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix},$$

with $\rho = 0.5$. We generate $n = 1000$ data points with 10 changepoints. Finally, set $\alpha = 0.05$ and $\delta = 0.05$.

```r
# Set parameters
n <- 1000
n_chg <- 10
delta <- 0.05
alpha <- 0.05
rho <- 0.5 # correlation among signal dimensions
s <- 1 # variance of signal components
Sigma <- matrix(c(s, rho, rho, s), nrow=2)
possible_states <- c(0, 2, -2)
```

2.2 Get changepoints

Randomly choose `n_chg=10` changepoints, under the constraint that they are spaced at least $n\delta$.

```r
# Randomly choose changepoints
set.seed(1112)
n_segments <- n_chg + 1
min_length <- ceiling(delta * n)

# Give every segment the minimum required length, then randomly distribute
# the remaining observations among the segments.
remaining_length <- n - n_segments * min_length

segment_lengths <- min_length +
  as.vector(
    rmultinom(
      n = 1,
      size = remaining_length,
      prob = rep(1, n_segments)
    )
  )

# Each changepoint is the last observation of a segment.
true_chg <- cumsum(segment_lengths)[-n_segments]

true_chg
```

```
##  [1]  94 190 280 375 465 546 639 730 821 910
```

```
segment_lengths
```

```
##  [1] 94 96 90 95 90 81 93 91 91 89 90
```

2.3 Generate data

Generate the $K \times 2$ matrix of means, with the constraint that the mean changes at each changepoint.

```
true_means <- matrix(
  NA_real_,
  nrow = n_segments,
  ncol = 2
)

true_means[1, ] <- sample(
  possible_states,
  size = 2,
  replace = TRUE
)

for (i in 2:n_segments) {
  true_means[i, 1] <- sample(
    setdiff(possible_states, true_means[i - 1, 1]),
    size = 1
  )

  true_means[i, 2] <- sample(
    setdiff(possible_states, true_means[i - 1, 2]),
    size = 1
  )
}

colnames(true_means) <- c("mu1", "mu2")

true_means
```

```
##       mu1 mu2
##  [1,]   2   2
##  [2,]   0   0
##  [3,]  -2  -2
##  [4,]   2   0
##  [5,]  -2   2
##  [6,]   2  -2
##  [7,]  -2   0
##  [8,]   0   2
##  [9,]  -2  -2
## [10,]   2   0
## [11,]  -2   2
```

Generate data:

```
set.seed(1418)
boundaries <- c(0, true_chg, n)

x <- matrix(
  NA_real_,
```

```
  nrow = n,
  ncol = 2
)

for (i in seq_len(n_segments)) {
  current_rows <- (boundaries[i] + 1):boundaries[i + 1]

  x[current_rows, ] <- rmvnorm(
    n = length(current_rows),
    mean = true_means[i, ],
    sigma = Sigma
  )
}

colnames(x) <- c("Y1", "Y2")
```

2.4 PoSIRd confidence regions

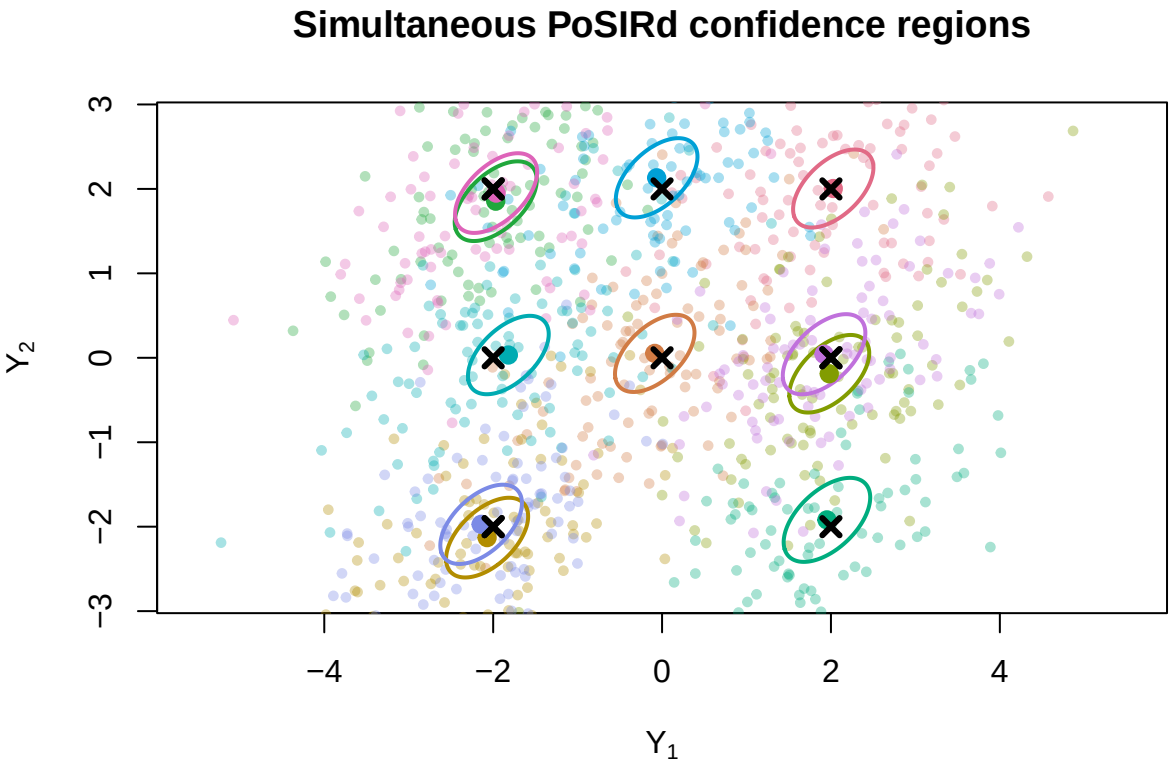
See the PoSIRd confidence regions with the given true changepoints

```
# See posir result with true segmentation
regions <- confidence_regions_1D(
  x = x,
  breakpoints = true_chg,
  delta = delta,
  alpha = alpha,
  Qtable = Qtable
)

mu1_grid <- seq(-2.8, 2.8, length.out = 300)
mu2_grid <- seq(-2.8, 2.8, length.out = 300)

plot(
  regions,
  mu1_grid = mu1_grid,
  mu2_grid = mu2_grid,
  main = "Simultaneous PoSIRd confidence regions",
  show_labels = FALSE
)

points(
  true_means[, 1],
  true_means[, 2],
  pch = 4,
  col = "black",
  lwd = 3,
  cex = 1.2
)
```



2.5 Confidence regions based on usual chi2 quantile

```
n_segments <- length(true_chg) + 1

naive_quantile <- sqrt(
  qchisq(
    p = 1 - alpha,
    df = 2
  )
)

bonferroni_quantile <- sqrt(
  qchisq(
    p = 1 - alpha / n_segments,
    df = 2
  )
)

naive_quantile
```

```
## [1] 2.447747
```

```
bonferroni_quantile
```

```
## [1] 3.284396
```

```
regions$quantile
```

```
## [1] 4.49
```

Create copies of the PoSIRd confidence-region object, replacing only the quantile

```r
regions_naive <- regions
regions_naive$quantile <- naive_quantile

regions_bonferroni <- regions
regions_bonferroni$quantile <- bonferroni_quantile
```

Visualization:

```r
old_par <- par(
  mfrow = c(1,3),
  mar = c(4, 4, 3, 1)
)

plot(
  regions_naive,
  mu1_grid = mu1_grid,
  mu2_grid = mu2_grid,
  main = "Naive",
  show_labels = FALSE
)

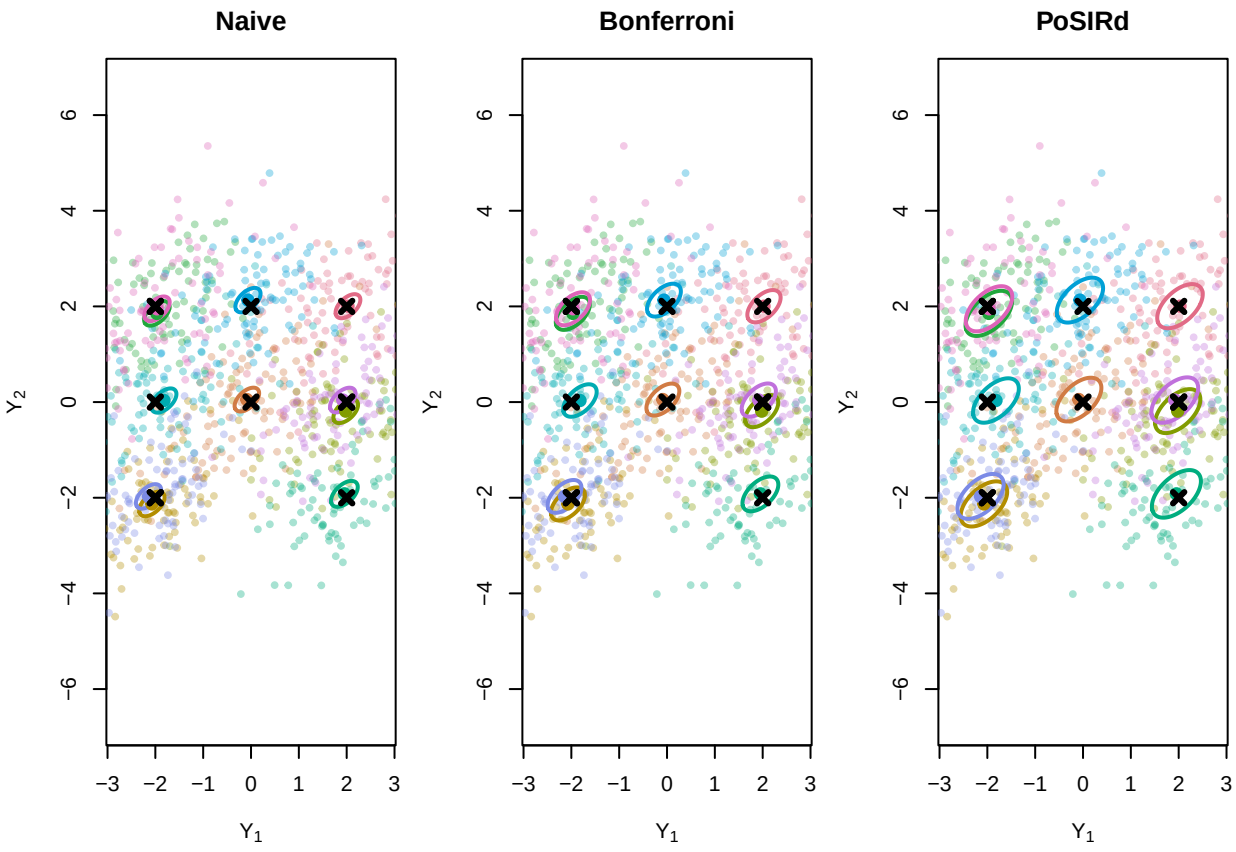
points(
  true_means[, 1],
  true_means[, 2],
  pch = 4,
  col = "black",
  lwd = 3,
  cex = 1.2
)

plot(
  regions_bonferroni,
  mu1_grid = mu1_grid,
  mu2_grid = mu2_grid,
  main = "Bonferroni",
  show_labels = FALSE
)

points(
  true_means[, 1],
  true_means[, 2],
  pch = 4,
  col = "black",
  lwd = 3,
  cex = 1.2
)

plot(
  regions,
  mu1_grid = mu1_grid,
  mu2_grid = mu2_grid,
  main = "PoSIRd",
  show_labels = FALSE
)
```

```
points(
  true_means[, 1],
  true_means[, 2],
  pch = 4,
  col = "black",
  lwd = 3,
  cex = 1.2
)
```




```
par(old_par)
```

2.6 Estimated changepoints

Let's now do the same with estimated changepoints

```
est_chg <- sort(doDynamicProgramming(x, K = n_chg + 5)$bkp)
if (length(est_chg)==length(true_chg))
  plot(est_chg, true_chg)

# ------------------------------------------------------------
# PoSIRd confidence regions
# ------------------------------------------------------------

regions_est <- confidence_regions_1D(
  x = x,
  breakpoints = est_chg,
  delta = delta,
```

```
  alpha = alpha,
  Qtable = Qtable
)

n_estimated_segments <- nrow(regions_est$means)

# ------------------------------------------------------------
# Naive and Bonferroni confidence regions
# ------------------------------------------------------------

naive_quantile <- sqrt(
  qchisq(
    p = 1 - alpha,
    df = 2
  )
)

bonferroni_quantile_est <- sqrt(
  qchisq(
    p = 1 - alpha / n_estimated_segments,
    df = 2
  )
)

regions_naive <- regions_est
regions_naive$quantile <- naive_quantile

regions_bonferroni <- regions_est
regions_bonferroni$quantile <- bonferroni_quantile_est

# ------------------------------------------------------------
# Construct the true signal at every observation
# ------------------------------------------------------------

true_signal <- matrix(
  NA_real_,
  nrow = n,
  ncol = 2
)

for (i in seq_len(n_segments)) {
  current_rows <- (boundaries[i] + 1):boundaries[i + 1]

  true_signal[current_rows, ] <- matrix(
    true_means[i, ],
    nrow = length(current_rows),
    ncol = 2,
    byrow = TRUE
  )
}

colnames(true_signal) <- c("mu1", "mu2")
# ------------------------------------------------------------
```

```r
# Compute the true target mean on each estimated segment
# ------------------------------------------------------------

estimated_boundaries <- c(
  0,
  est_chg,
  n
)

true_targets_est <- matrix(
  NA_real_,
  nrow = n_estimated_segments,
  ncol = 2
)

for (j in seq_len(n_estimated_segments)) {
  segment_indices <- (
    estimated_boundaries[j] + 1
  ):estimated_boundaries[j + 1]

  true_targets_est[j, ] <- colMeans(
    true_signal[
      segment_indices,
      ,
      drop = FALSE
    ]
  )
}

colnames(true_targets_est) <- c("mu1", "mu2")

# ------------------------------------------------------------
# Evaluate segmentwise coverage
# ------------------------------------------------------------

cvg_posir <- rep(
  NA,
  n_estimated_segments
)

cvg_naive <- rep(
  NA,
  n_estimated_segments
)

cvg_bonferroni <- rep(
  NA,
  n_estimated_segments
)

for (j in seq_len(n_estimated_segments)) {
  target_mean <- true_targets_est[j, ]
```

```
  cvg_posir[j] <- is_in_confidence_region(
    point = target_mean,
    region = regions_est,
    segment = j
  )

  cvg_naive[j] <- is_in_confidence_region(
    point = target_mean,
    region = regions_naive,
    segment = j
  )

  cvg_bonferroni[j] <- is_in_confidence_region(
    point = target_mean,
    region = regions_bonferroni,
    segment = j
  )
}

coverage_by_segment <- data.frame(
  segment = seq_len(n_estimated_segments),
  start = estimated_boundaries[-length(estimated_boundaries)] + 1,
  end = estimated_boundaries[-1],
  length = regions_est$lengths,
  target_mu1 = true_targets_est[, 1],
  target_mu2 = true_targets_est[, 2],
  valid = regions_est$valid,
  PoSIRd = cvg_posir,
  Naive = cvg_naive,
  Bonferroni = cvg_bonferroni
)

coverage_by_segment
```

```
##    segment start  end length target_mu1 target_mu2 valid PoSIRd Naive
## 1        1     1   94     94          2          2  TRUE   TRUE  TRUE
## 2        2    95  106     12          0          0 FALSE     NA    NA
## 3        3   107  190     84          0          0  TRUE   TRUE  TRUE
## 4        4   191  280     90         -2         -2  TRUE   TRUE  TRUE
## 5        5   281  342     62          2          0  TRUE   TRUE  TRUE
## 6        6   343  345      3          2          0 FALSE     NA    NA
## 7        7   346  375     30          2          0 FALSE     NA    NA
## 8        8   376  465     90         -2          2  TRUE   TRUE  TRUE
## 9        9   466  546     81          2         -2  TRUE   TRUE  TRUE
## 10      10   547  639     93         -2          0  TRUE   TRUE  TRUE
## 11      11   640  730     91          0          2  TRUE   TRUE  TRUE
## 12      12   731  821     91         -2         -2  TRUE   TRUE  TRUE
## 13      13   822  910     89          2          0  TRUE   TRUE  TRUE
## 14      14   911  932     22         -2          2 FALSE     NA    NA
## 15      15   933  955     23         -2          2 FALSE     NA    NA
## 16      16   956 1000     45         -2          2 FALSE     NA    NA
##    Bonferroni
## 1        TRUE
## 2          NA
```

```
## 3        TRUE
## 4        TRUE
## 5        TRUE
## 6          NA
## 7          NA
## 8        TRUE
## 9        TRUE
## 10       TRUE
## 11       TRUE
## 12       TRUE
## 13       TRUE
## 14         NA
## 15         NA
## 16         NA
```

```
# ------------------------------------------------------------
# Evaluate simultaneous coverage
# ------------------------------------------------------------


simultaneous_coverage <- c(
    PoSIRd = all(cvg_posir[regions_est$valid]),
    Naive = all(cvg_naive[regions_est$valid]),
    Bonferroni = all(cvg_bonferroni[regions_est$valid])
)


simultaneous_coverage
```

```
##     PoSIRd      Naive Bonferroni
##       TRUE       TRUE       TRUE
```

Visualization:

```
# ------------------------------------------------------------
# Plot the three methods
# ------------------------------------------------------------

old_par <- par(
  mfrow = c(1, 3),
  mar = c(4, 4, 3, 1)
)

plot(
  regions_naive,
  mu1_grid = mu1_grid,
  mu2_grid = mu2_grid,
  main = "Naive",
  show_labels = FALSE
)

points(
  true_targets_est[, 1],
  true_targets_est[, 2],
  pch = 4,
  col = "black",
```

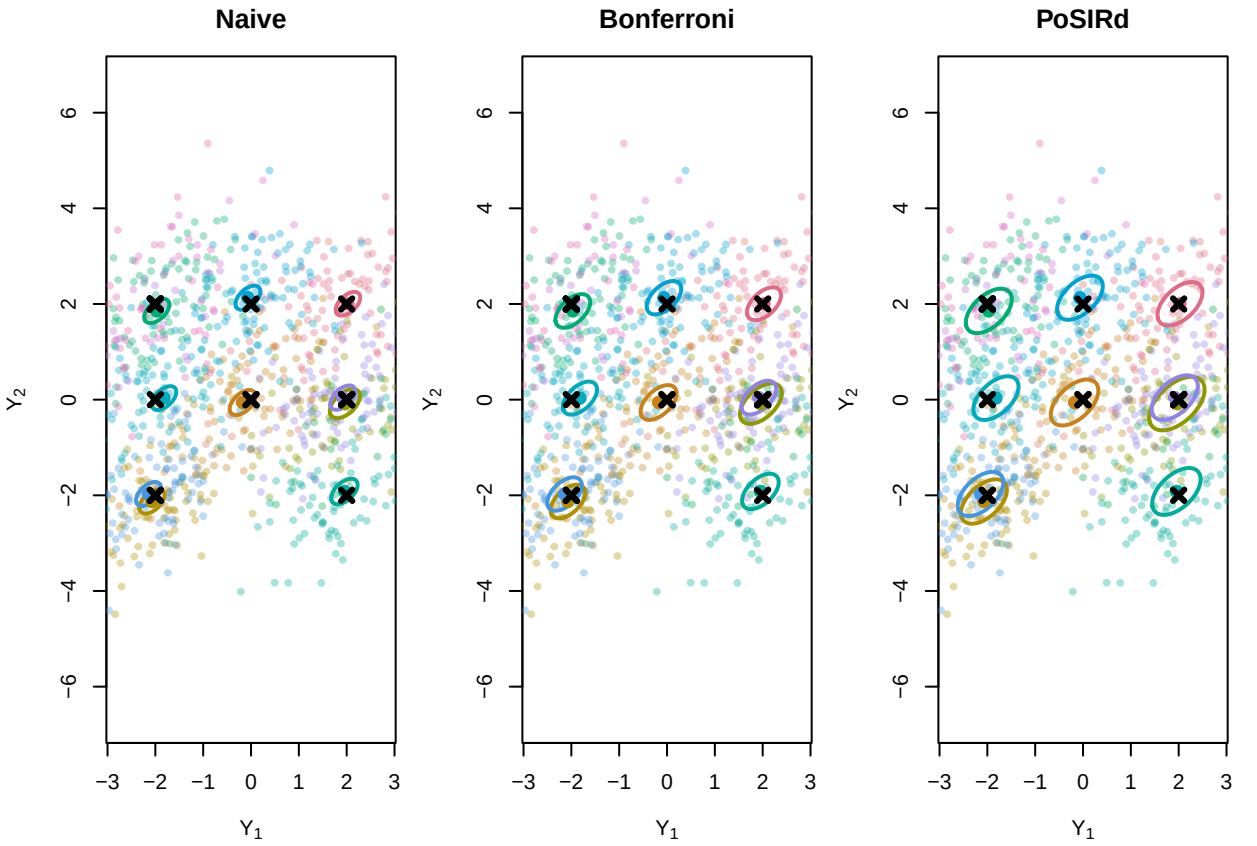
```
  lwd = 3,
  cex = 1.2
)

plot(
  regions_bonferroni,
  mu1_grid = mu1_grid,
  mu2_grid = mu2_grid,
  main = "Bonferroni",
  show_labels = FALSE
)

points(
  true_targets_est[, 1],
  true_targets_est[, 2],
  pch = 4,
  col = "black",
  lwd = 3,
  cex = 1.2
)

plot(
  regions_est,
  mu1_grid = mu1_grid,
  mu2_grid = mu2_grid,
  main = "PoSIRd",
  show_labels = FALSE
)

points(
  true_targets_est[, 1],
  true_targets_est[, 2],
  pch = 4,
  col = "black",
  lwd = 3,
  cex = 1.2
)
```




```
par(old_par)
```

3 Monte Carlo evaluation of coverage

Let's now proceed with a MC evaluation for the coverage in the same setting, by using the true changepoints. Since the changepoints are given, I expect the naive confidence region to undercover, and the posird and Bonferroni corrected ones to instead have the right coverage.

```
#-------------------------------------------------------------------------------
# Monte Carlo simulations, coverage evaluation, comparison with naive confidence
# regions
#-------------------------------------------------------------------------------
n_sim <- 1000
set.seed(1521)

simultaneous_coverage <- matrix(
  FALSE,
  nrow = n_sim,
  ncol = 3
)

colnames(simultaneous_coverage) <- c(
  "PoSIRd",
  "Naive",
  "Bonferroni"
)
```

```r
segment_coverage_posir <- matrix(
  FALSE,
  nrow = n_sim,
  ncol = n_segments
)

segment_coverage_naive <- matrix(
  FALSE,
  nrow = n_sim,
  ncol = n_segments
)

segment_coverage_bonferroni <- matrix(
  FALSE,
  nrow = n_sim,
  ncol = n_segments
)

areas_posir <- matrix(
  NA_real_,
  nrow = n_sim,
  ncol = n_segments
)

areas_naive <- matrix(
  NA_real_,
  nrow = n_sim,
  ncol = n_segments
)

areas_bonferroni <- matrix(
  NA_real_,
  nrow = n_sim,
  ncol = n_segments
)

for (sim in seq_len(n_sim)) {

  # Generate a new dataset with the same signal
  x <- matrix(
    NA_real_,
    nrow = n,
    ncol = 2
  )

  for (i in seq_len(n_segments)) {
    current_rows <- (boundaries[i] + 1):boundaries[i + 1]

    x[current_rows, ] <- rmvnorm(
      n = length(current_rows),
      mean = true_means[i, ],
      sigma = Sigma
    )
```

```
}

colnames(x) <- c("Y1", "Y2")

# PoSIRd regions
regions <- confidence_regions_1D(
  x = x,
  breakpoints = true_chg,
  delta = delta,
  alpha = alpha,
  Qtable = Qtable
)

# Critical values
posir_critical <- regions$quantile^2

naive_critical <- qchisq(
  1 - alpha,
  df = 2
)

bonferroni_critical <- qchisq(
  1 - alpha / n_segments,
  df = 2
)

# Coverage
for (i in seq_len(n_segments)) {
  distance_squared <- mahalanobis(
    x = matrix(true_means[i, ], nrow = 1),
    center = regions$means[i, ],
    cov = regions$covariance
  )

  statistic <- regions$lengths[i] * distance_squared

  segment_coverage_posir[sim, i] <-
    statistic <= posir_critical

  segment_coverage_naive[sim, i] <-
    statistic <= naive_critical

  segment_coverage_bonferroni[sim, i] <-
    statistic <= bonferroni_critical
}

simultaneous_coverage[sim, ] <- c(
  all(segment_coverage_posir[sim, ]),
  all(segment_coverage_naive[sim, ]),
  all(segment_coverage_bonferroni[sim, ])
)

# Ellipse areas
```

```
  covariance_factor <- sqrt(
    det(regions$covariance)
  )

  areas_posir[sim, ] <- pi *
    posir_critical /
    regions$lengths *
    covariance_factor

  areas_naive[sim, ] <- pi *
    naive_critical /
    regions$lengths *
    covariance_factor

  areas_bonferroni[sim, ] <- pi *
    bonferroni_critical /
    regions$lengths *
    covariance_factor

  if (sim %% 100 == 0) {
    print(sim)
  }
}
```

```
## [1] 100
## [1] 200
## [1] 300
## [1] 400
## [1] 500
## [1] 600
## [1] 700
## [1] 800
## [1] 900
## [1] 1000
```

```
colMeans(simultaneous_coverage)
```

```
##     PoSIRd      Naive Bonferroni
##      1.000      0.570      0.951
```

```
colMeans(segment_coverage_posir)
```

```
##  [1] 1 1 1 1 1 1 1 1 1 1 1 1
```

```
colMeans(segment_coverage_naive)
```

```
##  [1] 0.957 0.953 0.945 0.953 0.944 0.947 0.942 0.955 0.947 0.958 0.956
```

```
colMeans(segment_coverage_bonferroni)
```

```
##  [1] 0.996 0.997 0.998 0.995 0.997 0.993 0.992 0.997 0.997 0.997 0.991
```

```
area_summary <- data.frame(
  method = c(
    "PoSIRd",
    "Naive",
    "Bonferroni"
```

```
  ),
  mean_area = c(
    mean(areas_posir),
    mean(areas_naive),
    mean(areas_bonferroni)
  ),
  median_area = c(
    median(areas_posir),
    median(areas_naive),
    median(areas_bonferroni)
  )
)

area_summary
```

```
##       method mean_area median_area
## 1     PoSIRd 0.6035963   0.5999480
## 2      Naive 0.1793853   0.1783011
## 3 Bonferroni 0.3229719   0.3210198
```

As expected, the naive coverage is below the nominal level. The posird si conservative. This is not surprising since we are evaluating only the coverage over a limited number of intervals. The Bonferroni corrected one instead covers as expected. Let's do the same with estimated changepoints:

```
#-------------------------------------------------------------------------------
# Monte Carlo simulations, coverage evaluation, comparison with naive confidence
# regions
#-------------------------------------------------------------------------------
n_sim <- 1000
set.seed(1528)

simultaneous_coverage <- matrix(
  FALSE,
  nrow = n_sim,
  ncol = 3
)

colnames(simultaneous_coverage) <- c(
  "PoSIRd",
  "Naive",
  "Bonferroni"
)

segment_coverage_posir <- matrix(
  FALSE,
  nrow = n_sim,
  ncol = n_segments
)

segment_coverage_naive <- matrix(
  FALSE,
  nrow = n_sim,
  ncol = n_segments
)
```

```r
segment_coverage_bonferroni <- matrix(
  FALSE,
  nrow = n_sim,
  ncol = n_segments
)

areas_posir <- matrix(
  NA_real_,
  nrow = n_sim,
  ncol = n_segments
)

areas_naive <- matrix(
  NA_real_,
  nrow = n_sim,
  ncol = n_segments
)

areas_bonferroni <- matrix(
  NA_real_,
  nrow = n_sim,
  ncol = n_segments
)

for (sim in seq_len(n_sim)) {

  # Generate a new dataset with the same signal
  x <- matrix(
    NA_real_,
    nrow = n,
    ncol = 2
  )

  for (i in seq_len(n_segments)) {
    current_rows <- (boundaries[i] + 1):boundaries[i + 1]

    x[current_rows, ] <- rmvnorm(
      n = length(current_rows),
      mean = true_means[i, ],
      sigma = Sigma
    )
  }

  colnames(x) <- c("Y1", "Y2")

  est_chg <- sort(doDynamicProgramming(x, K = n_chg)$bkp)

  # PoSIRd regions
  regions <- confidence_regions_1D(
    x = x,
    breakpoints = est_chg,
    delta = delta,
    alpha = alpha,
```

```
  Qtable = Qtable
)

# Critical values
posir_critical <- regions$quantile^2

naive_critical <- qchisq(
  1 - alpha,
  df = 2
)

bonferroni_critical <- qchisq(
  1 - alpha / n_segments,
  df = 2
)

# Coverage
for (i in seq_len(n_segments)) {
  distance_squared <- mahalanobis(
    x = matrix(true_means[i, ], nrow = 1),
    center = regions$means[i, ],
    cov = regions$covariance
  )

  statistic <- regions$lengths[i] * distance_squared

  segment_coverage_posir[sim, i] <-
    statistic <= posir_critical

  segment_coverage_naive[sim, i] <-
    statistic <= naive_critical

  segment_coverage_bonferroni[sim, i] <-
    statistic <= bonferroni_critical
}

simultaneous_coverage[sim, ] <- c(
  all(segment_coverage_posir[sim, ]),
  all(segment_coverage_naive[sim, ]),
  all(segment_coverage_bonferroni[sim, ])
)

# Ellipse areas
covariance_factor <- sqrt(
  det(regions$covariance)
)

areas_posir[sim, ] <- pi *
  posir_critical /
  regions$lengths *
  covariance_factor

areas_naive[sim, ] <- pi *
```

```
    naive_critical /
    regions$lengths *
    covariance_factor

  areas_bonferroni[sim, ] <- pi *
    bonferroni_critical /
    regions$lengths *
    covariance_factor

  if (sim %% 100 == 0) {
    print(sim)
  }
}
```

```
## [1] 100
## [1] 200
## [1] 300
## [1] 400
## [1] 500
## [1] 600
## [1] 700
## [1] 800
## [1] 900
## [1] 1000
```

```
colMeans(simultaneous_coverage)
```

```
##     PoSIRd      Naive Bonferroni
##      0.999      0.563      0.951
```

```
colMeans(segment_coverage_posir)
```

```
##  [1] 1.000 1.000 1.000 1.000 1.000 1.000 1.000 1.000 1.000 1.000 0.999
```

```
colMeans(segment_coverage_naive)
```

```
##  [1] 0.948 0.952 0.956 0.948 0.952 0.946 0.945 0.947 0.950 0.943 0.942
```

```
colMeans(segment_coverage_bonferroni)
```

```
##  [1] 0.993 0.996 0.993 0.996 0.993 0.995 0.995 0.995 0.997 0.997 0.997
```

```
area_summary <- data.frame(
  method = c(
    "PoSIRd",
    "Naive",
    "Bonferroni"
  ),
  mean_area = c(
    mean(areas_posir),
    mean(areas_naive),
    mean(areas_bonferroni)
  ),
  median_area = c(
    median(areas_posir),
    median(areas_naive),
    median(areas_bonferroni)
```

```
  )
)

area_summary
```

```
##       method mean_area median_area
## 1     PoSIRd 0.6032103   0.5997814
## 2      Naive 0.1792706   0.1782515
## 3 Bonferroni 0.3227654   0.3209307
```

Finally, let's generate data under the null hypothesis that there are no mean change, but still require to have one changepoint. Write an alternative function that does not find a changepoint that is before $n\delta$ o $n - n\delta$, to have segments with the right length.

```
n_sim <- 2000

simultaneous_coverage <- matrix(
  FALSE,
  nrow = n_sim,
  ncol = 3
)

colnames(simultaneous_coverage) <- c(
  "PoSIRd",
  "Naive",
  "Bonferroni"
)

areas <- matrix(
  NA_real_,
  nrow = n_sim,
  ncol = 3
)

colnames(areas) <- c(
  "PoSIRd",
  "Naive",
  "Bonferroni"
)

selected_changepoints <- integer(n_sim)

select_dp_changepoint_constrained <- function(x, delta) {
  n <- nrow(x)
  min_length <- ceiling(delta * n)

  candidate_chg <- min_length:(n - min_length)

  costs <- vapply(
    candidate_chg,
    function(t) {
      x1 <- x[1:t, , drop = FALSE]
      x2 <- x[(t + 1):n, , drop = FALSE]

      mean1 <- colMeans(x1)
```

```
      mean2 <- colMeans(x2)

      sum(sweep(x1, 2, mean1, "-")^2) +
        sum(sweep(x2, 2, mean2, "-")^2)
    },
    numeric(1)
  )

  candidate_chg[which.min(costs)]
}
```

```
set.seed(1530)

for (sim in seq_len(n_sim)) {

  # ------------------------------------------------------------
  # Generate data with no true changepoints
  # ------------------------------------------------------------

  x <- rmvnorm(
    n = n,
    mean = c(0, 0),
    sigma = Sigma
  )

  colnames(x) <- c("Y1", "Y2")

  # ------------------------------------------------------------
  # Select the apparently strongest changepoint
  # ------------------------------------------------------------

  est_chg <- select_dp_changepoint_constrained(
    x = x,
    delta = delta
  )

  selected_changepoints[sim] <- est_chg

  # ------------------------------------------------------------
  # Confidence regions on selected intervals
  # ------------------------------------------------------------

  regions_est <- confidence_regions_1D(
    x = x,
    breakpoints = est_chg,
    delta = delta,
    alpha = alpha,
    Qtable = Qtable
  )

  critical_posir <- regions_est$quantile^2

  critical_naive <- qchisq(
    1 - alpha,
```

```
  df = 2
)

critical_bonferroni <- qchisq(
  1 - alpha / 2,
  df = 2
)

coverage_posir <- logical(2)
coverage_naive <- logical(2)
coverage_bonferroni <- logical(2)

for (i in 1:2) {
  distance_squared <- mahalanobis(
    x = matrix(c(0, 0), nrow = 1),
    center = regions_est$means[i, ],
    cov = regions_est$covariance
  )

  statistic <- regions_est$lengths[i] *
    distance_squared

  coverage_posir[i] <-
    statistic <= critical_posir

  coverage_naive[i] <-
    statistic <= critical_naive

  coverage_bonferroni[i] <-
    statistic <= critical_bonferroni
}

simultaneous_coverage[sim, ] <- c(
  all(coverage_posir),
  all(coverage_naive),
  all(coverage_bonferroni)
)

# ------------------------------------------------------------
# Mean ellipse area across the two selected segments
# ------------------------------------------------------------

covariance_factor <- sqrt(
  det(regions_est$covariance)
)

mean_inverse_length <- mean(
  1 / regions_est$lengths
)

areas[sim, ] <- pi *
  c(
    critical_posir,
```

```
      critical_naive,
      critical_bonferroni
    ) *
    covariance_factor *
    mean_inverse_length

  if (sim %% 100 == 0) {
    print(sim)
  }
}
```

```
## [1] 100
## [1] 200
## [1] 300
## [1] 400
## [1] 500
## [1] 600
## [1] 700
## [1] 800
## [1] 900
## [1] 1000
## [1] 1100
## [1] 1200
## [1] 1300
## [1] 1400
## [1] 1500
## [1] 1600
## [1] 1700
## [1] 1800
## [1] 1900
## [1] 2000
```

```
colMeans(simultaneous_coverage)
```

```
##     PoSIRd      Naive Bonferroni
##     0.9975     0.5365     0.6865
```